

如何使用 Cloud Build，自動將 GitHub 中的變更部署至 Cloud Run

來源：<https://codelabs.developers.google.com/codelabs/how-to-deploy-github-cloud-run-using-cloud-build?hl=zh-tw>

步驟數：12

如何使用 Cloud Build 將 GitHub 中的變更自動部署至 Cloud Run

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 啟用 API 並設定環境變數](#)
4. [4. 建立服務帳戶](#)
5. [5. 建立及設定 Firebase 專案](#)
6. [6. 編寫應用程式](#)
7. [7. 在本機執行應用程式](#)
8. [8. 建立 GitHub 存放區](#)
9. [9. 設定持續部署](#)
10. [10. 修正錯誤](#)
11. [11. 恭喜！](#)
12. [12. 清除所用資源](#)

步驟 1 · 約 0 分鐘

1. 簡介

總覽

在本程式碼研究室中，您將設定 Cloud Run，在您將原始碼變更推送到 GitHub 存放區時，[自動建構及部署](#)應用程式的新版本。

這個示範應用程式會將使用者資料儲存至 Firestore，但只有部分資料會正確儲存。您將設定持續部署，以便在將錯誤修正推送至 GitHub 存放區時，自動看到修正內容出現在新修訂版本中。

課程內容

- 使用 Cloud Shell 編輯器編寫 Express 網頁應用程式
 - 將 GitHub 帳戶連結至 Google Cloud，以便持續部署
 - 自動將應用程式部署至 Cloud Run
 - 瞭解如何使用 HTMX 和 TailwindCSS
-

步驟 2 · 約 0 分鐘

2. 設定和需求

必要條件

- 您擁有 GitHub 帳戶，且熟悉如何建立程式碼並推送至存放區。
- 您已登入 Cloud Console。
- 您先前已部署 Cloud Run 服務。舉例來說，您可以按照[從原始碼部署網路服務的快速入門導覽課程](#)，開始使用 Cloud Run。

啟用 Cloud Shell

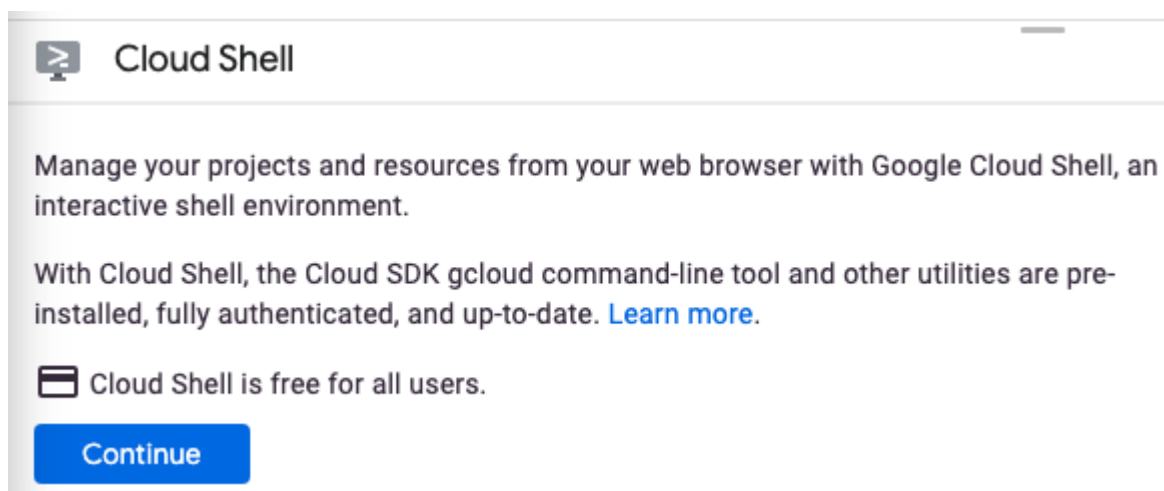
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



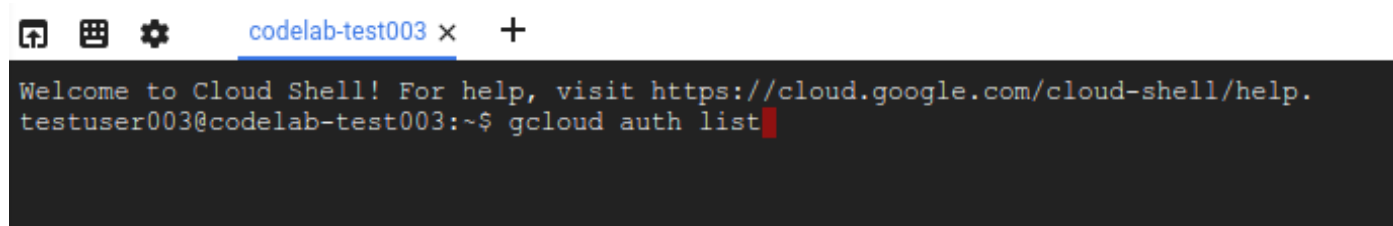
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

```
Credentialed Accounts
ACTIVE  ACCOUNT
*       <my_account>@<my_domain.com>

To set the active account, run:
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 0 分鐘

3. 啟用 API 並設定環境變數

啟用 API

本程式碼研究室需要使用下列 API。執行下列指令即可啟用這些 API：

```
gcloud services enable run.googleapis.com \  
  cloudbuild.googleapis.com \  
  firestore.googleapis.com \  
  iamcredentials.googleapis.com
```

設定環境變數

您可以設定本程式碼實驗室全程都會使用的環境變數。

```
REGION=<YOUR-REGION>  
PROJECT_ID=<YOUR-PROJECT-ID>  
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID --  
format='value(projectNumber)')  
SERVICE_ACCOUNT="firestore-accessor"  
SERVICE_ACCOUNT_ADDRESS=$SERVICE_ACCOUNT@$PROJECT_ID.iam.gserviceaccount.com
```

步驟 4 · 約 0 分鐘

4. 建立服務帳戶

Cloud Run 會使用這個服務帳戶呼叫 Vertex AI Gemini API。這個服務帳戶也會具備 Firestore 的讀取和寫入權限，以及從 Secret Manager 讀取密鑰的權限。

首先，請執行下列指令來建立服務帳戶：

```
gcloud iam service-accounts create $SERVICE_ACCOUNT \  
  --display-name="Cloud Run access to Firestore"
```

接著，授予服務帳戶 Firestore 的讀取及寫入權限。

```
gcloud projects add-iam-policy-binding $PROJECT_ID \  
  --member serviceAccount:$SERVICE_ACCOUNT_ADDRESS \  
  --role=roles/datastore.user
```

5. 建立及設定 Firebase 專案

1. 在 [Firebase 控制台](#) 中，按一下「新增專案」。
 2. 輸入 <YOUR_PROJECT_ID>，將 Firebase 新增至現有的其中一個 Google Cloud 專案
 3. 如果系統顯示提示，請詳閱並接受 [Firebase 條款](#)。
 4. 按一下「繼續」。
 5. 按一下「確認方案」，確認 Firebase 計費方案。
 6. 您可以選擇是否為這個程式碼研究室啟用 Google Analytics。
 7. 按一下「新增 Firebase」。
 8. 專案建立完成後，請按一下「繼續」。
 9. 在「建構」選單中，按一下「Firestore 資料庫」。
 10. 按一下 [Create database] (建立資料庫)。
 11. 從「位置」下拉式選單中選擇區域，然後按一下「下一步」。
 12. 使用預設的「以正式版模式啟動」，然後按一下「建立」。
-

步驟 6 · 約 0 分鐘

6. 編寫應用程式

首先，請建立原始碼的目錄，然後 `cd` 到該目錄。

```
mkdir cloud-run-github-cd-demo && cd $_
```

接著，請建立 `package.json` 檔案，並加入以下內容：

```
{
  "name": "cloud-run-github-cd-demo",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "node app.js",
    "nodemon": "nodemon app.js",
    "tailwind-dev": "npx tailwindcss -i ./input.css -o ./public/output.css --watch",
    "tailwind": "npx tailwindcss -i ./input.css -o ./public/output.css",
    "dev": "npm run tailwind && npm run nodemon"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "@google-cloud/firestore": "^7.3.1",
    "axios": "^1.6.7",
    "express": "^4.18.2",
    "htmx.org": "^1.9.10"
  },
  "devDependencies": {
    "nodemon": "^3.1.0",
    "tailwindcss": "^3.4.1"
  }
}
```

首先，請使用以下內容建立 `app.js` 來源檔案。這個檔案包含服務的進入點，以及應用程式的主要邏輯。

```

const express = require("express");
const app = express();
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
const path = require("path");
const { get } = require("axios");

const { Firestore } = require("@google-cloud/firestore");
const firestoreDb = new Firestore();

const fs = require("fs");
const util = require("util");
const { spinnerSvg } = require("./spinnerSvg.js");

const service = process.env.K_SERVICE;
const revision = process.env.K_REVISION;

app.use(express.static("public"));

app.get("/edit", async (req, res) => {
  res.send(`<form hx-post="/update" hx-target="this" hx-swap="outerHTML">
    <div>
      <p>
        <label>Name</label>
        <input class="border-2" type="text" name="name" value="Cloud">
      </p><p>
        <label>Town</label>
        <input class="border-2" type="text" name="town" value="Nibelheim">
      </p>
    </div>
    <div class="flex items-center mr-[10px] mt-[10px]">
      <button class="btn bg-blue-500 text-white px-4 py-2 rounded-lg text-center text-sm font-medium mr-[10px]">Submit</button>
      <button class="btn bg-gray-200 text-gray-800 px-4 py-2 rounded-lg text-center text-sm font-medium mr-[10px]" hx-get="cancel">Cancel</button>
      ${spinnerSvg}
    </div>
  </form>`);
});

app.post("/update", async function (req, res) {
  let name = req.body.name;
  let town = req.body.town;
  const doc = firestoreDb.doc(`demo/${name}`);

  //TODO: fix this bug
  await doc.set({
    name: name
  });
});

```

```

    /* town: town */
  });

  res.send(`<div hx-target="this" hx-swap="outerHTML" hx-indicator="spinner">
    <p>
      <div><label>Name</label>: ${name}</div>
    </p><p>
      <div><label>Town</label>: ${town}</div>
    </p>
    <button
      hx-get="/edit"
      class="bg-blue-500 text-white px-4 py-2 rounded-lg text-sm font-
medium mt-[10px]"
    >
      Click to update
    </button>
  </div>`);
});

app.get("/cancel", (req, res) => {
  res.send(`<div hx-target="this" hx-swap="outerHTML">
    <p>
      <div><label>Name</label>: Cloud</div>
    </p><p>
      <div><label>Town</label>: Nibelheim</div>
    </p>
    <div>
      <button
        hx-get="/edit"
        class="bg-blue-500 text-white px-4 py-2 rounded-lg text-sm font-
medium mt-[10px]"
      >
        Click to update
      </button>
    </div>
  </div>`);
});

const port = parseInt(process.env.PORT) || 8080;
app.listen(port, async () => {
  console.log(`booth demo: listening on port ${port}`);

  //serviceMetadata = helper();
});

app.get("/helper", async (req, res) => {
  let region = "";
  let projectId = "";

```

```

let div = "";

try {
  // Fetch the token to make a GCF to GCF call
  const responsel = await get(
    "http://metadata.google.internal/computeMetadata/v1/project/project-id",
    {
      headers: {
        "Metadata-Flavor": "Google"
      }
    }
  );

  // Fetch the token to make a GCF to GCF call
  const response2 = await get(
    "http://metadata.google.internal/computeMetadata/v1/instance/region",
    {
      headers: {
        "Metadata-Flavor": "Google"
      }
    }
  );

  projectId = responsel.data;
  let regionFull = response2.data;
  const index = regionFull.lastIndexOf("/");
  region = regionFull.substring(index + 1);

  div = `
<div>
This created the revision <code>${revision}</code> of the
Cloud Run service <code>${service}</code> in <code>${region}</code>
for project <code>${projectId}</code>.
</div>`;
} catch (ex) {
  // running locally
  div = `<div> This is running locally.</div>`;
}

res.send(div);
});

```

建立名為 `spinnerSvg.js` 的檔案

```

module.exports.spinnerSvg = `

```

為 tailwindCSS 建立 `input.css` 檔案

```

@tailwind base;
@tailwind components;
@tailwind utilities;

```

並為 tailwindCSS 建立 `tailwind.config.js` 檔案

```

/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ["./**/*.{html,js}"],
  theme: {
    extend: {}
  },
  plugins: []
};

```

並建立 `.gitignore` 檔案。

```
node_modules/  
  
npm-debug.log  
coverage/  
  
package-lock.json  
  
.DS_Store
```

現在，請建立新的 `public` 目錄。

```
mkdir public  
cd public
```

並在該公開目錄中，為前端建立 `index.html` 檔案，該檔案將使用 `htmx`。

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta
      name="viewport"
      content="width=device-width, initial-scale=1.0"
    />
    <script
      src="https://unpkg.com/htmx.org@1.9.10"
      integrity="sha384-
D1Kt99CQMDuVetoL1lrYwg5t+9QdHe7NLX/SoJYkXDFfX37iInKRY5xLSi8nO7UC"
      crossorigin="anonymous"
    ></script>

    <link href="./output.css" rel="stylesheet" />
    <title>Demo 1</title>
  </head>
  <body
    class="font-sans bg-body-image bg-cover bg-center leading-relaxed"
  >
    <div class="container max-w-[700px] mt-[50px] ml-auto mr-auto">
      <div class="hero flex items-center">
        <div class="message text-base text-center mb-[24px]">
          <h1 class="text-2xl font-bold mb-[10px]">
            It's running!
          </h1>
          <div class="congrats text-base font-normal">
            Congratulations, you successfully deployed your
            service to Cloud Run.
          </div>
        </div>
      </div>

      <div class="details mb-[20px]">
        <p>
          <div hx-trigger="load" hx-get="/helper" hx-swap="innerHTML" hx-
target="this">Hello</div>
        </p>
      </div>

      <p
        class="callout text-sm text-blue-700 font-bold pt-4 pr-6 pb-4 pl-10
leading-tight"
      >
        You can deploy any container to Cloud Run that listens for
        HTTP requests on the port defined by the
        <code>PORT</code> environment variable. Cloud Run will

```

```

        scale automatically based on requests and you never have to
        worry about infrastructure.
    </p>

    <h1 class="text-2xl font-bold mt-[40px] mb-[20px]">
        Persistent Storage Example using Firestore
    </h1>
    <div hx-target="this" hx-swap="outerHTML">
        <p>
            <div><label>Name</label>: Cloud</div>
        </p><p>
            <div><label>Town</label>: Nibelheim</div>
        </p>
        <div>
            <button
                hx-get="/edit"
                class="bg-blue-500 text-white px-4 py-2 rounded-lg text-sm font-
medium mt-[10px]"
            >
                Click to update
            </button>
        </div>
    </div>

    <h1 class="text-2xl font-bold mt-[40px] mb-[20px]">
        What's next
    </h1>
    <p class="next text-base mt-4 mb-[20px]">
        You can build this demo yourself!
    </p>
    <p class="cta">
        <button
            class="bg-blue-500 text-white px-4 py-2 rounded-lg text-center
text-sm font-medium"
        >
            VIEW CODELAB
        </button>
    </p>
</div>
</body>
</html>

```


7. 在本機執行應用程式

在本節中，您將在本機執行應用程式，確認使用者嘗試儲存資料時，應用程式會發生錯誤。

首先，您必須擁有 Datastore 使用者角色才能存取 Firestore (如果使用身分驗證，例如在 Cloud Shell 中執行)，或是模擬先前建立的使用者帳戶。

在本機執行時使用 ADC

如果您在 Cloud Shell 中執行，表示您已在 Google Compute Engine 虛擬機器上執行。[應用程式預設憑證 \(ADC\)](#) 會自動使用與這個虛擬機器相關聯的憑證 (如執行 `gcloud auth list` 所示)，因此不必使用 `gcloud auth application-default login` 指令。不過，您的身分仍需具備 Datastore 使用者角色。您可以跳到「在本機執行應用程式」一節。

不過，如果您是在本機終端機上執行 (即不在 Cloud Shell 中)，則需要使用應用程式預設憑證向 Google API 進行驗證。您可以 1) 使用憑證登入 (前提是您具有 Datastore 使用者角色)，或 2) 模擬本程式碼研究室中使用的服務帳戶登入。

選項 1) 使用 ADC 的憑證

如要使用憑證，請先執行 `gcloud auth list`，確認您在 `gcloud` 中的驗證方式。接著，您可能需要授予身分「Vertex AI 使用者」角色。如果您的身分具備「擁有者」角色，您就已擁有這個 Datastore 使用者角色。如果沒有，可以執行這項指令，將 Vertex AI 使用者角色和 Datastore 使用者角色授予您的身分。

```
USER=<YOUR_PRINCIPAL_EMAIL>

gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member user:$USER \
  --role=roles/datastore.user
```

然後執行下列指令

```
gcloud auth application-default login
```

選項 2) 模擬 ADC 的服務帳戶

如要使用本程式碼研究室建立的服務帳戶，您的使用者帳戶必須具備[服務帳戶權杖建立者角色](#)。您可以執行下列指令來取得這個角色：

```
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member user:$USER \
  --role=roles/iam.serviceAccountTokenCreator
```

接著，執行下列指令，搭配服務帳戶使用 ADC

```
gcloud auth application-default login --impersonate-service-
account=$SERVICE_ACCOUNT_ADDRESS
```

在本機執行應用程式

接著，請確認您位於程式碼研究室的根目錄 `cloud-run-github-cd-demo`。

```
cd .. && pwd
```

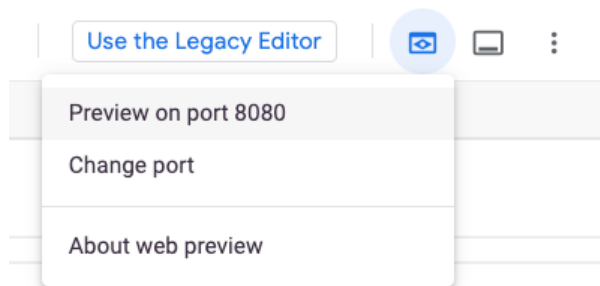
現在要安裝依附元件。

```
npm install
```

最後，您可以執行下列指令碼來啟動應用程式。這個指令碼也會從 `tailwindCSS` 產生 `output.css` 檔案。

```
npm run dev
```

現在開啟網路瀏覽器，前往 `http://localhost:8080`。如果您使用 Cloud Shell，可以開啟「網頁預覽」按鈕，然後選取「預覽通訊埠 8080」，即可開啟網站。



在姓名和城鎮輸入欄位中輸入文字，然後點選「儲存」。然後重新整理頁面。你會發現「town」欄位並未保留。您將在後續章節中修正這個錯誤。

停止在本機執行的 Express 應用程式 (例如在 macOS 上按下 `Ctrl^c`)。

步驟 8 · 約 0 分鐘

8. 建立 GitHub 存放區

注意：如果這是您第一次在 Cloud Shell 中設定 Git，請先設定 Git [使用者名稱](#)和[電子郵件地址](#)，再繼續操作。

您可以執行 `git config --global user.name` 確認目前的 Git 使用者名稱，並執行 `git config --global user.email` 確認目前的電子郵件地址

在本地目錄中，建立以 main 為預設分支名稱的新存放區。

```
git init
git branch -M main
```

提交含有錯誤的目前程式碼集。設定持續部署功能後，您將修正錯誤。

```
git add .
git commit -m "first commit for express application"
```

前往 GitHub，建立私人或公開的空存放區。本程式碼研究室建議將存放區命名為 `cloud-run-auto-deploy-codelab`。如要建立空白存放區，請取消勾選所有預設設定或設為「無」，這樣建立存放區時，預設不會有任何內容，例如：

Initialize this repository with:

☐ Add a README file

This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None ▾

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

Create repository

如果正確完成這個步驟，空白存放區頁面會顯示下列操作說明：

Quick setup — if you've done this kind of thing before

 Set up in Desktop or **HTTPS** **SSH** `https://github.com/` 

Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# deleteme1234" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/
git push -u origin main
```

...or push an existing repository from the command line

```
git remote add origin https://github.com/
git branch -M main
git push -u origin main
```

...or import code from another repository

You can initialize this repository with code from a Subversion, Mercurial, or TFS project.

[Import code](#)

您將按照「從指令列推送現有存放區」的指示，執行下列指令：

首先，請執行以下指令新增遠端存放區：

```
git remote add origin <YOUR-REPO-URL-PER-GITHUB-INSTRUCTIONS>
```

然後將主要分支版本推送至上游存放區。

```
git push -u origin main
```

注意：如果系統提示輸入使用者名稱和密碼，建議您建立個人存取權杖，以便[透過 Cloud Shell 向 Git 進行驗證](#)。如要瞭解如何建立 GitHub 的個人存取權杖，請參閱[這篇文章](#)。

授予個人存取權杖「Repo」範圍。這樣一來，您就能從 Cloud Shell 將變更推送到存放區。然後在系統提示輸入使用者名稱和密碼時，使用個人存取權杖做為密碼。

9. 設定持續部署

現在您已在 GitHub 中擁有程式碼，可以設定持續部署。前往 [Cloud Run 的 Cloud 控制台](#)。

- 按一下「建立服務」
- 按一下「從存放區持續部署」
- 按一下「設定 Cloud Build」。
- 在「原始碼存放區」下方
 - 選取 GitHub 做為存放區供應商
 - 按一下「管理已連結的存放區」，設定 Cloud Build 對存放區的存取權
 - 選取存放區，然後按一下「下一步」
- 在「Build Configuration」(版本設定) 底下
 - 將「Branch」(分支版本) 保留為 `^main$`
 - 在「建構類型」中，選取「Go、Node.js、Python、Java、.NET Core、Ruby 或 PHP (透過 Google Cloud 的 Buildpacks 使用)」
- 將「建構作業的結構定義目錄」保留為 `/`
- 按一下「儲存」。
- 在「驗證」下方
 - 按一下「允許未經驗證的叫用」
- 在「容器、磁碟區、網路、安全性」下方
 - 在「安全性」分頁中，選取您在先前步驟中建立的服務帳戶，例如 `Cloud Run access to Firestore`。
- 按一下 [Create]

這會部署含有錯誤的 Cloud Run 服務，您將在下一節修正錯誤。

步驟 10 · 約 0 分鐘

10. 修正錯誤

修正程式碼中的錯誤

在 Cloud Shell 編輯器中開啟 `app.js` 檔案，然後前往標示為 `//TODO: fix this bug` 的註解。

將下列行從

```
//TODO: fix this bug
  await doc.set({
    name: name
  });
```

到

```
//fixed town bug
  await doc.set({
    name: name,
    town: town
  });
```

執行以下指令，驗證修正內容：

```
npm run start
```

並開啟網路瀏覽器。再次儲存該鎮的資料，然後重新整理。重新整理後，您會看到新輸入的城鎮資料已正確保留。

確認修正內容後，即可部署。首先，請提交修正內容。

```
git add .
git commit -m "fixed town bug"
```

然後推送到 GitHub 的上游存放區。

```
git push origin main
```

Cloud Build 會自動部署變更。您可以前往 Cloud Run 服務的 Cloud 控制台，監控部署變更。

在正式環境中驗證修正內容

如果 Cloud Run 服務的 Cloud Console 顯示第 2 個修訂版本目前提供 100% 的流量 (例如 https://console.cloud.google.com/run/detail/<YOUR_REGION>/<YOUR_SERVICE_NAME>/revisions)，您可以在瀏覽器中開啟 Cloud Run 服務網址，並確認在重新整理頁面後，新輸入的城鎮資料是否仍存在。

步驟 11 · 約 0 分鐘

11. 恭喜！

恭喜您完成本程式碼研究室！

建議您參閱 [Cloud Run](#) 和從 [Git](#) 持續部署的說明文件。

涵蓋內容

- 使用 Cloud Shell 編輯器編寫 Express 網頁應用程式
 - 將 GitHub 帳戶連結至 Google Cloud，以便持續部署
 - 自動將應用程式部署至 Cloud Run
 - 瞭解如何使用 HTMX 和 TailwindCSS
-

12. 清除所用資源

為避免產生意外費用 (例如，Cloud Run 服務意外叫用次數超過[免費層級的每月 Cloud Run 叫用次數配額](#))，您可以刪除 Cloud Run，或刪除您在步驟 2 中建立的專案。

如要刪除 Cloud Run 服務，請前往 <https://console.cloud.google.com/run> 的 Cloud Run Cloud 控制台，然後刪除您在本程式碼實驗室中建立的 Cloud Run 服務，例如刪除 `cloud-run-auto-deploy-codelab` 服務。

如要刪除整個專案，請前往 <https://console.cloud.google.com/cloud-resource-manager>，選取您在步驟 2 中建立的專案，然後選擇「刪除」。刪除專案後，您必須在 Cloud SDK 中變更專案。如要查看所有可用專案的清單，請執行 `gcloud projects list`。
